**问题背景**： EC600MCNLA-自动开机电路 BOOT33 传参到 APP 侧。具体即在 boot 33 下面使用 ql_gpio_get_level 读取 GPIO 电平，如何把读取到的 gpio 电平传递到 app 侧?

**实现原理**：首先找到一块合适的 RAM 区域，此区域为代码运行的各个阶段所共享；将需要传递的参数写入这块区域，在需要访问的时候再读出。

Boot33/logo 阶段：asr_property_set()/asr_property_get()

App 阶段：ql_asr_property_set()/ql_asr_property_get()

**实现举例**：以 GPIO_PIN_NO_31 脚为例，实现在 logo 侧 GPIO 电平的读取，并通过 RAM 将电平值传递到 app 侧

1. 在 F:\depot3\ASR\3601\dev\R06\startup\logo\src\main.c 文件目录下找到 logo()函数（**logo 函数为 logo 阶段运行的第一个函数**）并进行如下操作

```
void logo(void)
{
    unsigned int val;
    volatile UINT8 var;
    unsigned int i;
    UINT32 battery_level;
    //[1] uart init
    cp_uart_init();
    CP_LOGD("LOGO START\r\n");
    usticaInit();

    /*[2] Print LOGO HEADER INFO */
    CP_LOGW("[LOGO] VERSION_DATE   :[%s]\r\n",(logo_vb.vb.logo_version_block.version_date));
    CP_LOGD("LCD_TYPE:[%s]\r\n",logo_vb.vb.logo_version_block.oem_lcd_type);
    CP_LOGD("ver:%s\r\n",logo_vb.vb.logo_version_block.build_info_string);
    CP_LOGD("LOGO_FLAG:%s\r\n",logo_vb.FeedbackBlock.feedback_block.LogoDisplayLogoFlag);

    asr_property_set("logo.version",logo_vb.vb.logo_version_block.build_info_string);
    asr_property_set("logo.lcd_type",logo_vb.vb.logo_version_block.oem_lcd_type);
    asr_property_set("logo.displayflag",logo_vb.FeedbackBlock.feedback_block.LogoDisplayLogoFlag);

    asr_property_dump();

    ql_gpio_init(GPIO_PIN_NO_31, PIN_DIRECTION_OUT, PIN_PULL_PU,  PIN_LEVEL_HIGH);
    PIN_LEVEL_E  value;
    ql_gpio_get_level(GPIO_PIN_NO_31,&value);
    asr_property_set("gpio_no31_level",&value);

    //[3] Enable MPU and using the cortex-r default memory map as the background region
    val = sctlr_get();
    val |= SCTLR_BR | SCTLR_M;
    sctlr_set(val);

    //[4] MPU config
    CP_LOGD("logo MPU config\r\n");
    set_mpu_reg_0();
    set_mpu_reg_3();
    set_mpu_reg_4();
    set_mpu_for_DSP();
//  mpu_value_check();
//    PMIC_Init();
//  sdcard_init();

#ifdef DEBUG_FLOAT_OPERATOR
    /* Note :can execute float operation but can not printf float type data with %f */
    CP_LOGD("test_float_data!\r\n");
    test_float_data();
    test_fitting_calculate();
#endif

#ifdef SUPPORT_MALLOC
   /* support malloc mechanism */
   malloc_init(g_MallocBuffer, MALLOC_BUFFER_SIZE);
   CP_LOGD("malloc init done!\r\n");
#endif

   /* [5] ptable init and dump */
   if(ptable_init()){
```

将31脚初始化为高电平，调用ql_gpio_get_level接口读取电平信号，通过asr_property_set()接口实现电平信号的保存

2. 在 app 侧 example_gpio.c 中调用 ql_asr_property_get（）接口进行 logo 侧保存的电平值得获取

```
#ifndef QUEC_ARR_SIZE
#define QUEC_ARR_SIZE(a)            ( sizeof(a) / sizeof(a[0]) )
#endif

static void quec_test_ram_property(void * argv)
{
    char  *level = NULL;
    level = ql_asr_property_get("gpio_no31_level");
    test_log("boot33 readlevel:%d\r\n",*level);
}

static quec_gpio_cfg_t __quec_gpio_cfg[] =
{
    {GPIO_PIN_NO_31, PIN_DIRECTION_OUT, PIN_NO_EDGE, PIN_PULL_DISABLE, PIN_LEVEL_LOW},
    {GPIO_PIN_NO_32, PIN_DIRECTION_OUT, PIN_NO_EDGE, PIN_PULL_DISABLE, PIN_LEVEL_LOW},
#if 0 ...
#endif
};

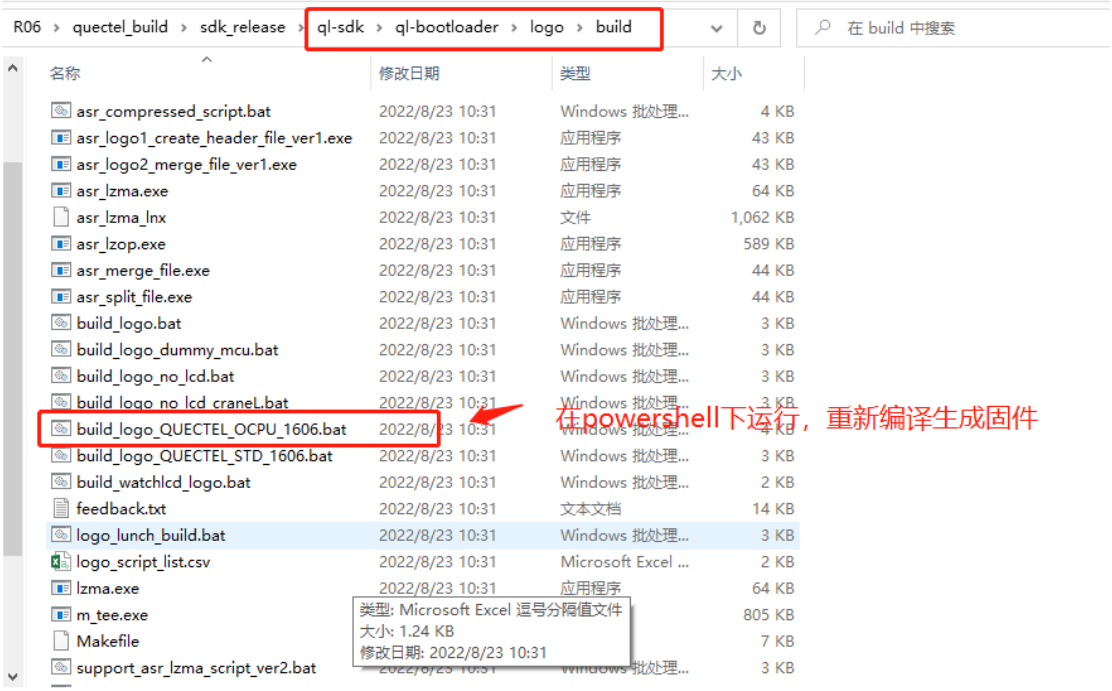
static QlOSStatus __quec_gpio_cust_config(void) ...

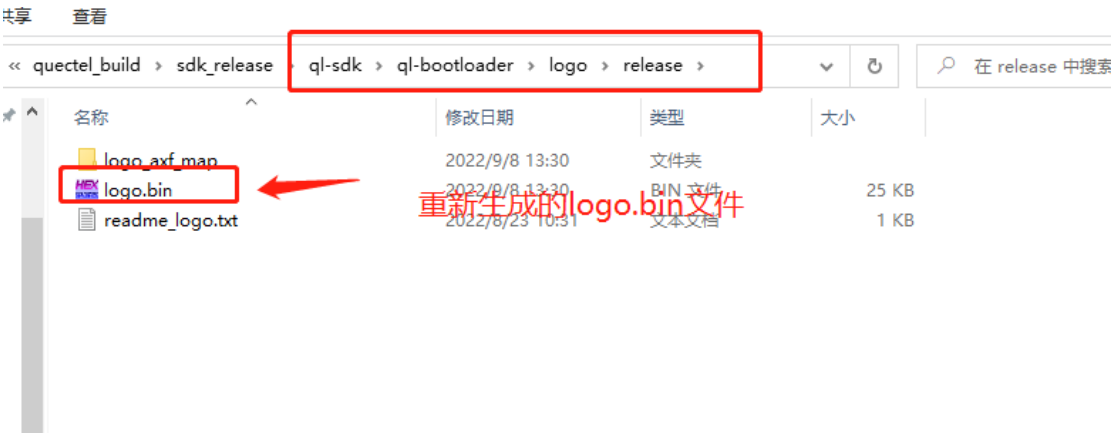
static void quec_gpio_test(void * argv) ...

#endif

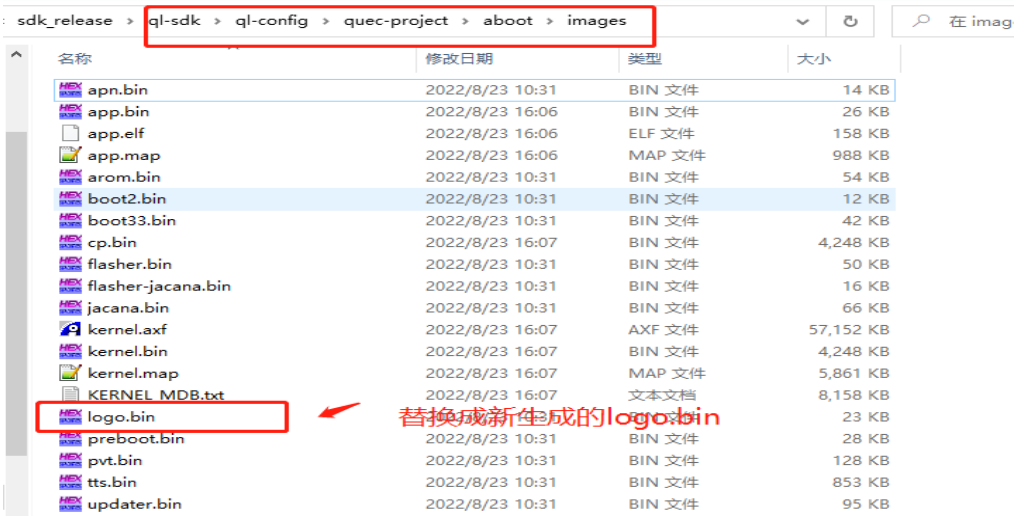
application_init(quec_test_ram_property, "quec_test_ram_property", 2, 0);
```

3. 重新编译生成新的 logo.bin 并替换掉原来的 logo.bin



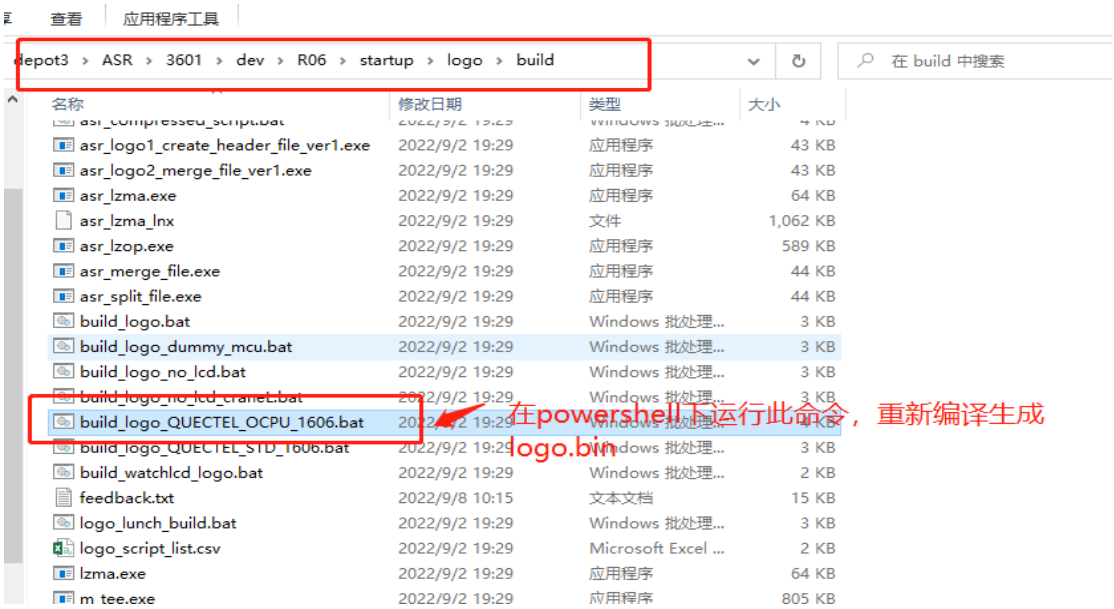

在 ql-sdk\ql-config\quec-project\aboot\images 下进行替换



4. 依次编译 app、firmware 指令，重新生成固件即可。

**移远内部版本**

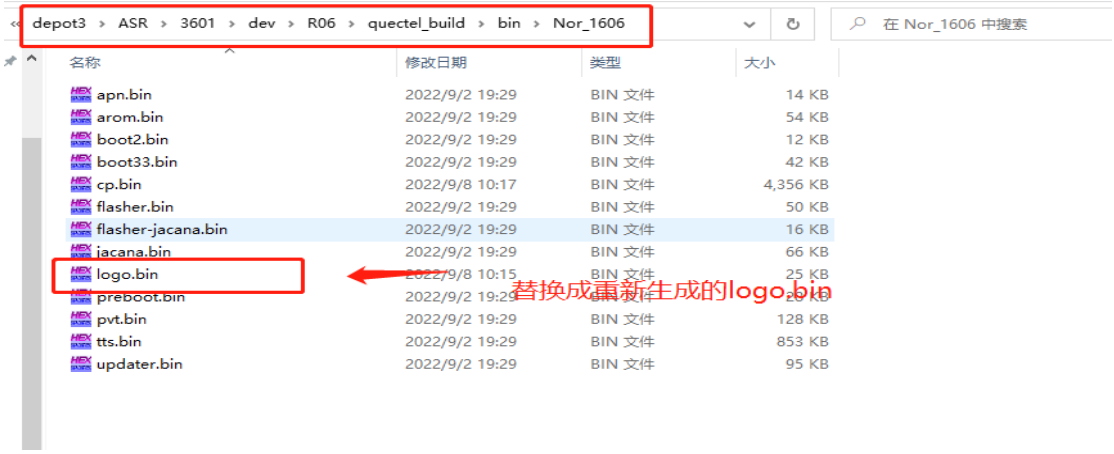
重新编译生成新的 logo.bin 并替换掉原来的 logo.bin





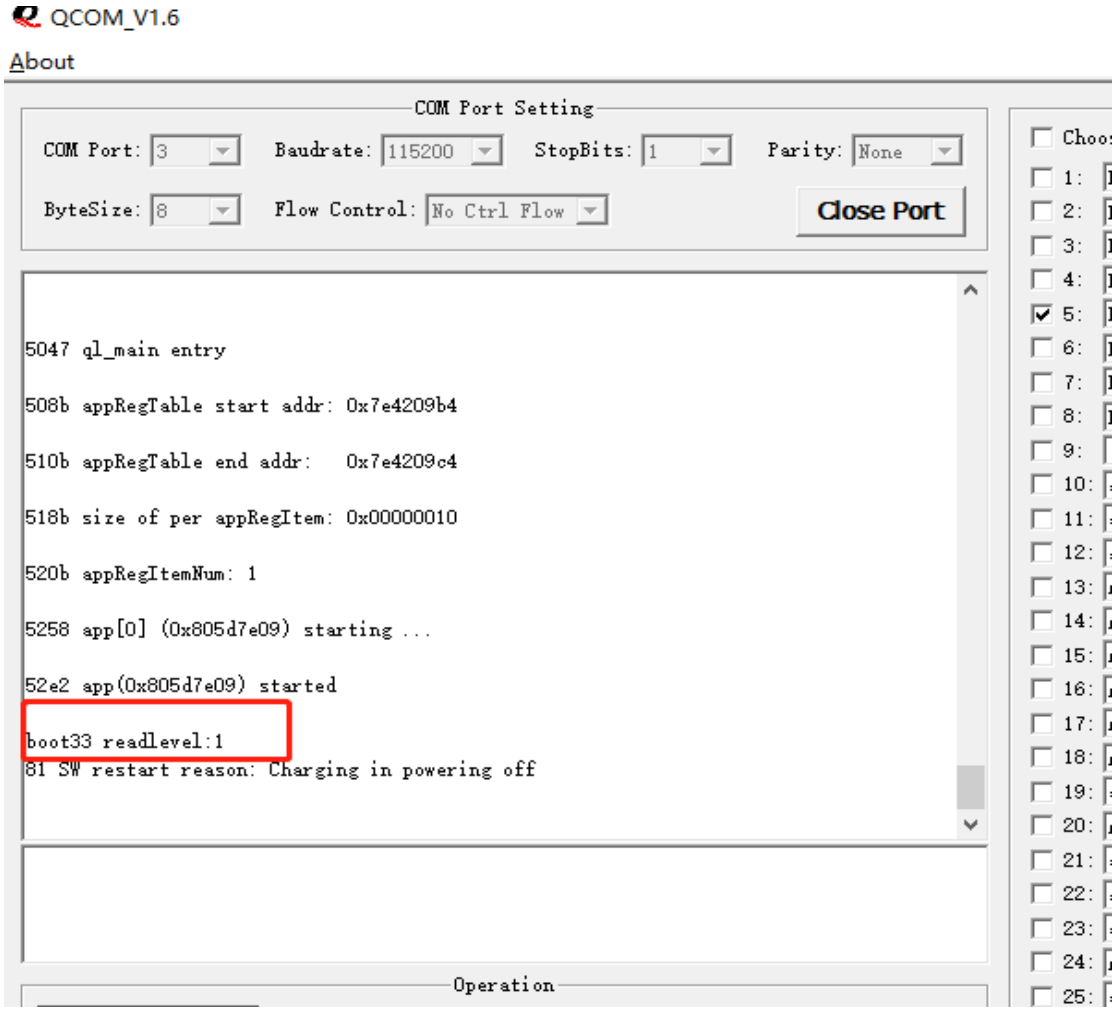


在 F:\depot3\ASR\3601\dev\R06\quectel_build\bin\Nor_1606 下进行替换




重新编译下 buildlibbyOPTfile.bat 即可。

结果如下：




成功实现 logo 侧的 GPIO 引脚电平通过 RAM 传递到 app 侧。